Lab 4: Microcontroller Programming in C

ECE 212: Digital Circuits II – Fall 2025

Abstract. This lab will introduce you to programming in C on a PIC32 microcontroller. The PIC32 is a small MIPS microprocessor that includes a number of useful peripherals and I/O interfaces. In this lab, we will program the PIC32 to interact with the and control the LafBot V2 robot, shown below.

The LafBot is equipped with two analog optical sensors that can differentiate between a white and black surface under the front of the robot, allowing it to follow a black line. At the end of this lab, we will program the LafBot to follow a line using these built-in sensors, by telling the LafBot to turn based on which sensor is detecting the line.



Objectives.

- Program a microcontroller in C and interface with I/O peripherals
- Practice C programming constructs like if-statements, loops, and function calls

Materials and Equipment.

- MPLAB X Lab IDE v 6.02
- LafBot V2 equipped with PIC32 microcontroller

Deliverables.

- In-person lab demonstration (per team) for each programming component
- Code submission via GitHub repository
- Lab notebook via Moodle (per team)

Part 0: Set Up a MPLAB Project

Goal: To program devices with the MIPS Assembly Language, we will use a development environment called MPLAB X IDE v6.20. Follow the steps below to set up a new MPLAB project for this lab assignment in your team's GitHub repository.

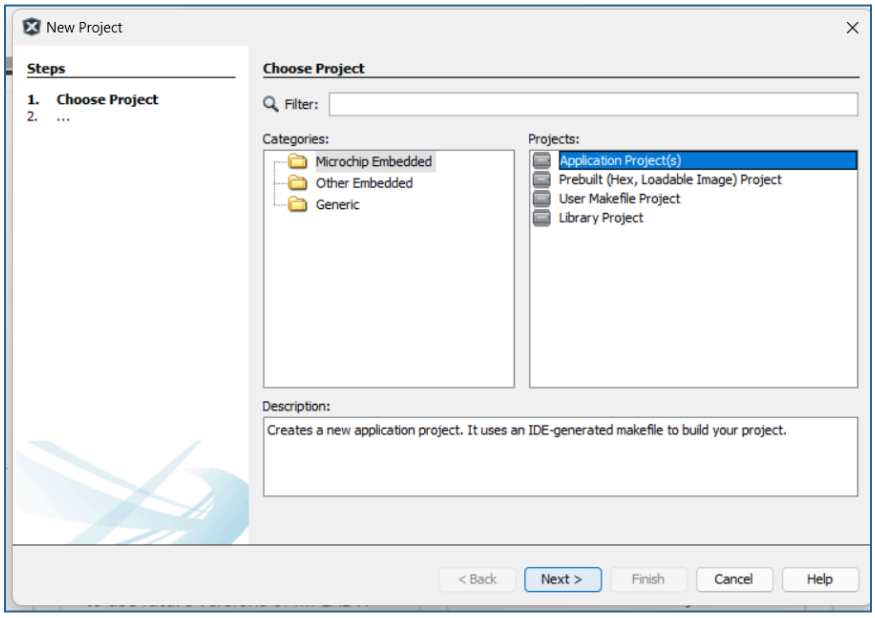
0.1. Make a new folder titled `Lab_4` inside of your team's local GitHub repository (a folder named `ece212_team_name`). You should save all of your source files inside of this folder and/or the entire MPLAB project.

0.2. Open **MPLAB X IDE v6.20** on the lab machine.
Note: There are two versions of MPLAB on the computers, v6.20 and v6.25. Only v6.20 will work for this lab assignment. Make sure to select the correct version!

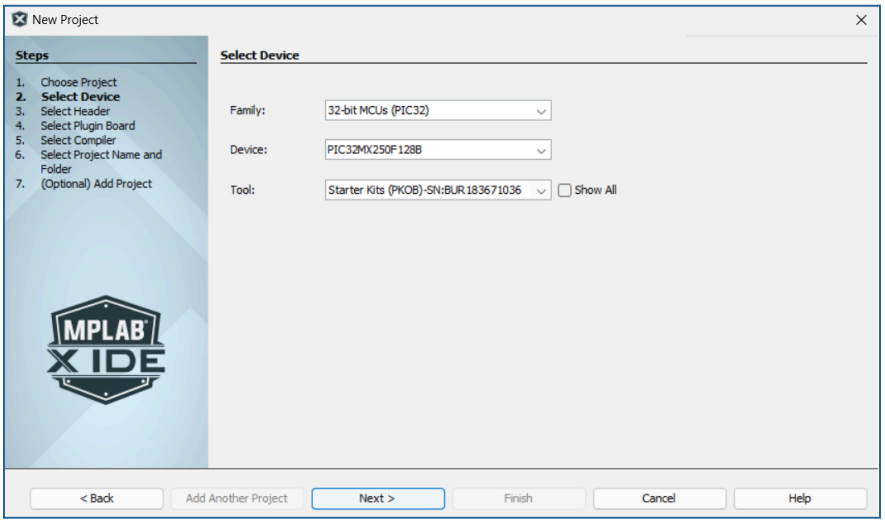
0.3. Create a new MPLAB project and configure it to work with the PIC32 microcontroller:

a. From the toolbar, select *File* then *New Project.*

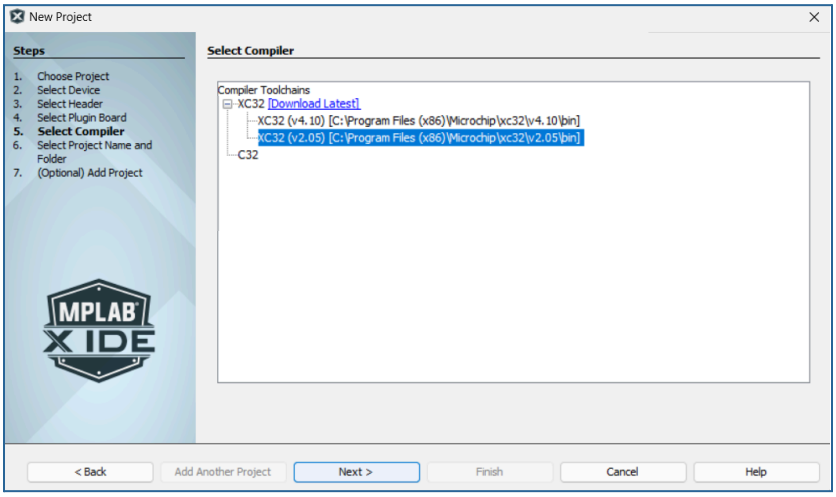
b. Under *Choose Project*, select the folder corresponding to *Microchip Embedded* then *Application Project(s)*.



c. Under *Select Device*, set the Family to *32-bit MCUs (PIC32)* and the Device to ***PIC32MX250F128B***. If the tool option *Starter Kids (PKOB)-SN:....* Appears, select that tool. Otherwise, choose the default (*No Tool)*.



d. Under, *Select Compiler,* Select the ***XC32 (v2.05)*** compiler

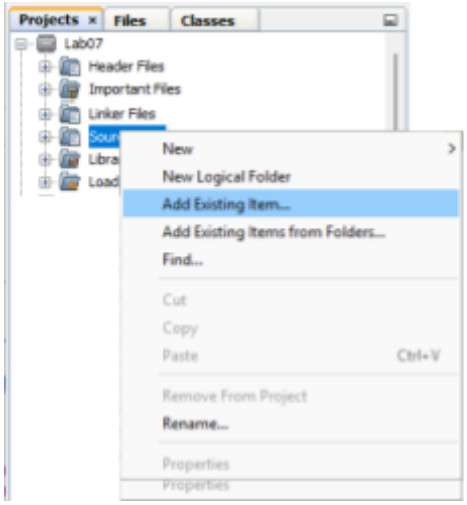


e. Finally, select the project folder to be the folder inside of your Git repository (`ece212_team_name`) and choose a name for your project.

f. Click Finish.

0.4. Your project will open as a collection of folders in the *Projects* panel on the left-side of the screen. In the following steps, you will add the provided C files to the project.

a. In the *Projects* panel, right click the folder *Source Files* and select *Add Existing Item.*



b. Select the provided starter code `lab4.c` from the file explorer. This file should be in your GitHub repository in order to ensure it is backed-up properly!

c. In the *Projects* panel, right click the folder *Header Files* and select *Add Existing Item.* Select the provided header file `ece212.h` from the file explorer.
Note that both the header file and the main file should exist in the same folder on your computer. Otherwise, you will encounter error messages when building your project!

0.5. Double click the imported file `lab4.c` to open it in the text editor!

Part 1: Run a Simple Function on the PIC32 Microcontroller

Goal: In the second part of the lab, you will run a simple C program on the PIC32 microcontroller. The purpose of this step is to familiarize you with workflow used in later parts of the lab.

In Part 0, you added the ***header file*** `ece212.h` to your C programming project. This file includes functions and variables that you will use in the lab to interface with the LafBot board. You are welcome to open this file and look at its contents, but you are not expected to understand every line of code. For Part 1 of this lab, you will use the following function found in `ece212.h`:

`void writeLEDs(int value)`	This function takes an input, `value`, that is a 4-bit hexadecimal number, and uses that number to turn a set of 4 LEDs on or off. If the least-significant bit of `value` is one, then LED 1 is turned on, otherwise it is off. If the next bit of `value` is one, then LED 2 is turned on, otherwise it is off, *etc.* For example, `writeLEDs(0xF)` would turn all four LEDs on.

1.1. Locate the following code in the C source file `lab4.c`:

```
void part1() {
   ece212_setup();
   while (1) {  // Repeats contents forever!
      writeLEDs(0xF);
   }
}
```

a. This code defines a ***function*** called `part1()` that includes some instructions. You will create a function for each part of this lab exercise. The contents of the function are located between the first pair of curly brackets ({ and }).

b. The first line of code in the function is a call to another function called `ece212_setup()`. This function is defined in `ece212.h` and configures the PIC32 microcontroller when the program begins running.

c. The next like of code, beginning with `while(1)` is a ***continuous loop***. The contents of this loop repeat forever. Having a continuous loop in a microcontroller program is common if you want your code to continue executing while the microcontroller is on.

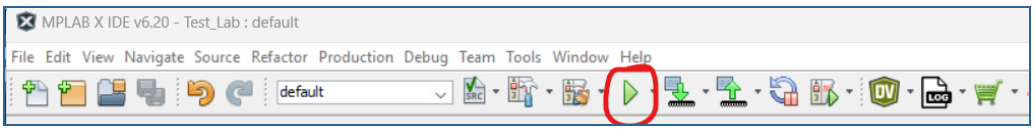
d. The `writeLEDs(0xF)` command inside the loop turns all the LafBot's LEDs on. Since this command is in the continuous loop, the LEDs will continue to be on until the microcontroller is turned off or unplugged.

1.2. Add a `main()` function in your C program that calls the `part1()` function. Recall that the `main()` function is the starting point when your C program runs. This is where your code begins executing.

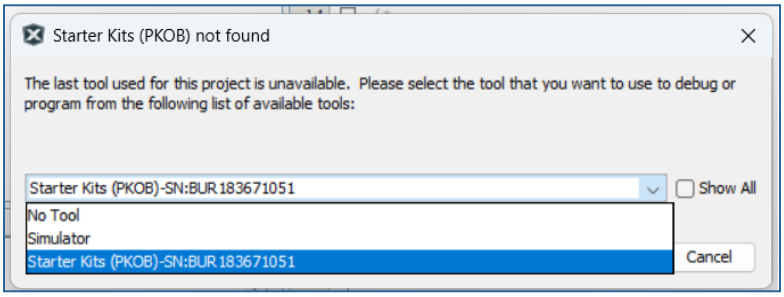
1.3. Upload your C program to the PIC32 Microcontroller to see how it runs on the LafBot board. You can program the PIC32 using the following steps:

a. Connect a mini USB cable to the Microstick and the USB end to the computer

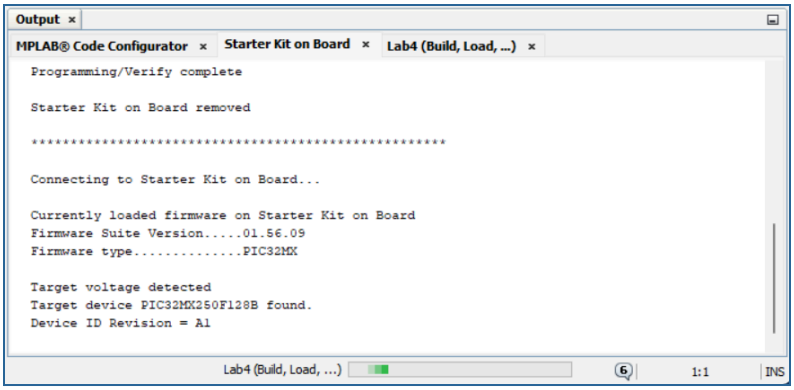
b. Select *Run Main Project* by clicking the "play" button in the top toolbar



c. When a window appears to select a starter kit, select the device under "*Starter Kids (PKOB)-SN:...*"



d. After selecting the starter kit, MPLAB will begin compiling your project and will automatically upload the compiled executable to the connected PIC32 board. You can monitor the progress in the *Output* panel at the bottom of the window. Once the upload has completed, you will see the message *"Programming/Verify complete."*



e. If you are having trouble programming your Microstick, check the following:

- Make sure the AB selector switch on the Microstick II is set to A. This sets the pins used to program the chip.
- After programming, press the ON button and make sure the LED D3 is on.
- The buttons only work when the LafBot is on. The LafBot automatically powers off after ~9 minutes.

➢ For this section, your lab notebook only needs to contain a copy of your current `main()` function implementation. No schematics or testing information is required.

Part 2: LafBot Hypnotizer

Goal: In Part 2 of this lab, you will alter the main function of your C program to create a "hypnotizer" blinking LED program that repeatedly flashes the LEDs, as described below..

To complete Part 2 of the lab exercise, you may need to use any of the previously described functions, as well as the following function found in `ece212.h`:

`void delayms(int time)`	This function takes an input, `time`, and pauses the execution of the program for that many milliseconds.

2.1. Create a new function in `lab4.c` called `part2()`.

2.2. Have your function call `ece212_setup()`. This function initializes the outputs used by the LafBot LEDs

2.3. Implement your function to meet the following specification: the program should alternate turning each of the four LafBot LEDs on. The first LED (**`RB0`**) should be on, then that LED should turn off and the next LED should turn on (**`RB1`**), *etc*. After reaching the last LED, the pattern should wrap around and begin again on the other side.

2.4. Modify the `main()` function to call your new function for `part2()`. Then, test your program by running it on the LafBot.

2.5. Demonstrate your working program to the instructor.

➢ For this section, your lab notebook should contain: The implementation of your `part2()` function. No testing information or diagrams are required.

Part 3: LafBot Start Your Engines

Goal: In Part 3 of this lab, you test controlling the LafBot motors. Your program should repeat the following logic: Move the LafBot forward in a straight line for 0.5 seconds; Wait one second; Move the LafBot backward in a straight line for 0.5 seconds; Wait one second. Your program should move the LafBot at half speed.

To complete Part 3 of the lab exercise, you may need to use any of the previously described functions, as well as the following variables found in `ece212.h`:

`RFOWARD`	A variable that controls the speed of the LafBot's *right* motor in the forward direction. This variable can be set to a 16-bit hexadecimal value, where assigning the value `0x0000` turns the motor off and `0xFFFF` turns the motor on at full speed.
`RBACK`	A variable that controls the speed of the LafBot's *right* motor in the backward (reverse) direction. This variable can be set to a 16-bit hexadecimal value, where assigning the value `0x0000` turns the motor off and `0xFFFF` turns the motor on (in reverse) at full speed.
`LFOWARD`	Works as described for `RFORWARD`, except controls the LafBot's *left* motor.
`LBACK`	Works as described for `RBACK`, except controls the LafBot's *left* motor.

3.1. Create a new function in `lab4.c` called `part3()`.

3.2. Have your function call `ece212_lafbot_setup()`. This function initializes the PWM outputs used by the LafBot motors

3.3. Implement your function to meet the following specification, that repeats forever until the LafBot is turned off: Move the LafBot forward in a straight line, at half speed, for 0.5 seconds; Wait one second; Move the LafBot backward in a straight line, at half speed for 0.5 seconds; Wait one second before repeating.

3.4. Modify `main()` to call your new function for `part3()`. Then, test your program by running it on the LafBot. To check that the LafBot is operating properly, ensure that the LafBot board is turned on. Then, press the button titled "*Motor Enable*." This will send power to the motors.

3.5. Demonstrate your working program to the instructor.

➢ For this section, your lab notebook should contain: The implementation of your `part3()` function. No testing information or diagrams are required.

Part 4: LafBot the Line-Following Robot

Goal: In Part 4 of this lab, you will write a C program for the PIC32 to control the LafBot so that it follows a black line on a white background over a predetermined course as quickly as possible. To do this, your program should read the value of the LafBot's sensors and decide whether to continue the LafBot moving straight, or turn right or left, or increase or decrease its speed.

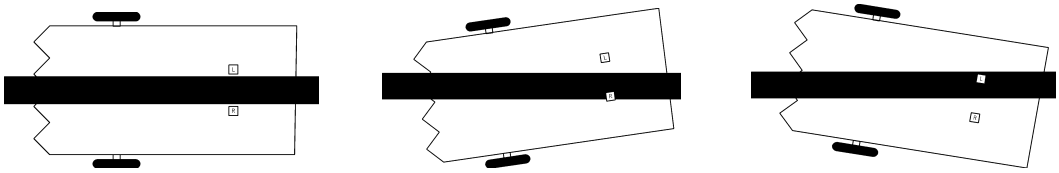
To complete Part 4 of the lab exercise, you may need to use any of the previously described functions, as well as the following functions and variables found in `ece212.h`:

`int analogRead(char pin)`	This function takes an input, `pin`, corresponding to one of the two sensors on the LafBot board (defined below). The function returns a 10-bit unsigned integer value that represents the amount of light reflected by the sensor. A small value indicates that the sensor is over a light-colored object, whereas a large value indicates that the sensor is over a dark-colored object. The `analogRead()` function takes the measured voltage, `m`, as a percentage of the maximum voltage (3.3V), and then scales this to a 10-bit unsigned integer value, according to the following formula. $m/3.3 * (2^{10}-1)$ For example, the minimum voltage (0V) maps to the integer 0 and the maximum voltage (3.3V) maps to the integer value 1023. The physical sensors give a voltage value between 0.8V to 1.2V, depending on the amount of light reflected, and the `analogRead()` function returns this voltage value scaled using the above equation. The sensor reading can differ based on the amount of ambient light in the room and whether the LafBot is powered by the USB cable or battery pack.
`LEFT_SENSOR`	A variable that contains the PIN of the left light sensor underneath the LafBot
`RIGHT_SENSOR`	A variable that contains the PIN of the right light sensor underneath the LafBot

4.1. Create a new function in `lab4.c` called `part4()`.

4.2. Have your function call `ece212_lafbot_setup()`. This function initializes the PWM outputs used by the LafBot motors

4.3. Implement your `part4()` function to meet the following specification, that repeats forever until the LafBot is turned off: ***Drive, following the black line based on the sensor readings!*** This is a very general goal. Here are some specifics you should consider for controlling your LafBot:



LafBot On Course *LafBot Too Far Right* *LafBot Too Far Left*

Figure 1. Steering the LafBot

a. The LafBot will initially be placed on the track so that it is "on course" and the two sensors straddle the black line. Their value should reflect seeing a light color.

b. If the LafBot is approaching a curve, its sensors will indicate that it is off course, either too far to the left or too far to the right (See Figure 1).

c. When off course, you can change the speed of the motors to turn the LafBot. For example, turning the left motor off and the right motor on will turn the LafBot to the left. You can also keep both motors on at some speed so the LafBot moves forward while also turning

d. When on course, you can have the LafBot's speed increase or decrease, rather than always moving at a constant speed, to win the race!

Additionally, consider the following when implementing your code:

a. You may find it useful to use LEDs `RB0` and `RB3` to indicate whether your software detects the black stripe or white background.

b. Your program should wait a short period (*e.g.,* 75 ms) after changing the LafBot's motor speed before reading the sensors again.

c. It is recommended that you update the motor speeds only ***once*** per loop iteration (i.e., once per time period). You can do this by assigning the motor values to separate, temporary variables within the loop. Then, at the end of the loop, use a single assignment for `LFORWARD` and `RFORWARD`.

4.4. Modify `main()` to call your new function for `part4()`. Then, test your program by running it on the LafBot using the sample track on your workbench. Once you are confident your LafBot can follow the sample track, "race" your LafBot on the large oval track in the center of the lab.

4.5. Demonstrate your working LafBot to the instructor and time how long it takes for your LafBot to navigate the large oval track.

4.6. Test your LafBot on the wacky track. Does it stay on the track? If so, how long does it take to finish? If not, what modifications could you make to your code for it to work on the wacky track?

➢ For this section, your lab notebook should contain: The implementation of your `part4()` function, results of timing your LafBot around the oval and wacky tracks (including multiple times if you conduct multiple trial runs), and the answers to the questions in Step 4.6.

Part 5: Commit Your Changes to GitHub

Goal: Throughout the lab exercise, you have made changes to files inside your `ece212_team_name` folder. In this step, you will sync these changes with GitHub's cloud. For more detailed instructions on using GitHub Desktop, please refer to *Lab 1*.

4.1. In GitHub Desktop, commit your changes by typing a commit message and selecting the button "*Commit X files to main"* in the left panel.

4.2. Select the button *"Push Origin"* under the title *Push commits to the origin remote.*

4.3. Verify that you can see your code on GitHub.com. Prevent data loss by always committing to GitHub before leaving the lab room, even if you are still working on your lab solution!

Lab Notebook *(Due before the next lab session)*

Lab Notebook. In collaboration with your lab partner, submit one lab notebook on Moodle that describes the lab exercise and your team's circuit design. Your lab notebook is worth 50% of your grade for each lab assignment. Your lab notebook should include the following sections:

1. **Heading/Title:** Your lab notebook should begin with a title block that includes the course number, the title of the lab assignment, the names of each team member, and the date of the lab assignment.

2. **Distribution of Work:** Briefly describe the contributions of each team member to the lab assignment, including writing of the lab notebook. What is specified in this section will not affect your grade, however it is expected that you periodically rotate all roles within your lab team if you do divide work.

3. **For each part of the lab exercise (*e.g.*, Part 1, Part 2), include the following:**

    a. **Design and Implementation:** Briefly describe the design of your circuit using equations, tables, and diagrams as appropriate. Diagrams can be hand-drawn as long as they are neat and detailed. You do not need to submit code as part of your lab notebook. Your code submission is done via GitHub! *If any part of the lab assignment indicates that you should record a block diagram, truth table, or equation, include those in this section.*

    b. **Testing:** Describe how you tested your circuit, including the specific test cases you used for each module and the result of each test. Record these tests as a table or waveform, as appropriate. If your test cases failed initially, record your failed test result, and briefly summarize the error once discovered. Include subsequent test results, as applicable. *If any part of the lab assignment indicates that you should create a testbench or conduct a test, include those results in this section.*

    c. **Questions:** A number of technical questions may be posed in each part of the lab assignment. If questions are given and they do not fit into the above categories, include your answers to them in this section. If no questions are given, you do not need to include this section.

4. **Discussion/Errata:** In the final section of your lab notebook, reflect on the lab exercise. What did you learn through completing this lab exercise? Did you encounter any errors when implementing your design? Make a note of what the error was and the solution or the correct SystemVerilog syntax.

Your lab notebook should be typed and submitted as a .pdf document. Only one member per team needs to submit the lab notebook.

Appendix A: Programming the PIC32 Microcontroller

Microchip Technology, Inc. markets a number of small microcontroller chips. Its PIC32 family of microcontrollers includes a processor based on the MIPS architecture along with a number of peripherals and I/O connections that are inexpensive (less than $5) and used in many embedded systems. The specific processor we will use is the **PIC32MX250F128B**.

Microchip also provides an integrated development environment for programming the chip called ***MPLAB X*** and a cross-compiler called ***XC32*** which allows C programs entered into the IDE to be compiled and downloaded into a PIC32 chip.

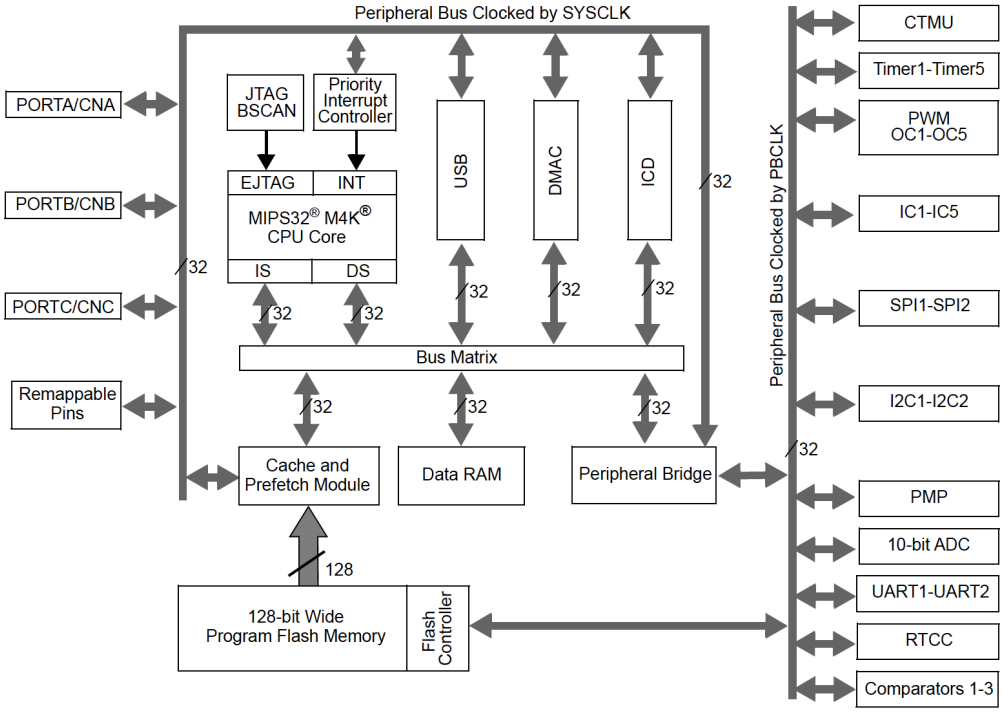


Figure A-1. PIC32 Block Diagram

Figure A-1 shows the overall organization of PIC32 chips, which includes the following features:

- A MIPS processor core
- Flash EEPROM for program storage (128KB for the PIC32MX250F128B)
- RAM for data storage (32KB for the PIC32MX250F128B)
- a USB Interface
- a 10-bit Analog-Digital Converter
- two UART modules for serial data transmission & reception
- several timer circuits to precisely measure timing intervals
- a Pulse-Width Modulation (PWM) unit
- I2C (Inter-Integrated Circuit) / SPI (Serial Peripheral Interface) unit
- Parallel ports that can be used for general purpose I/O (GPIO)using digital signals.

Microchip Technology also markets a small (20mm X 76mm) development board called the Microstick II, which includes a 28-pin PIC32 chip along with a USB connector that can be connected to a personal computer for programming (and optionally, power supply). Figure A-2 shows the Microstick II board, which includes a reset switch, a user LED that can be turned on and off under program control, and two 14-pin header sockets on the bottom of the board. These sockets can be attached using header pins to either a solderless breadboard or another printed circuit board with a matching header socket.

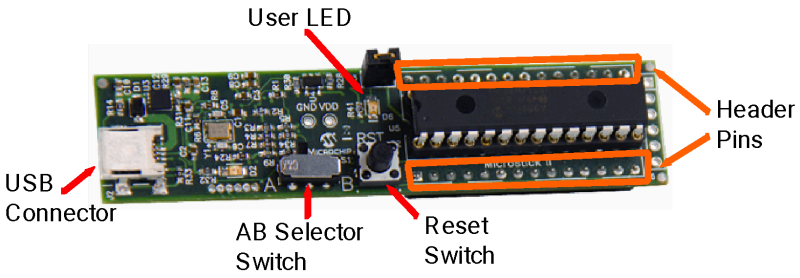


Figure A-2. Microstick II Development Board

LafBot V2 Peripherals.

The LafBot is equipped with two ***analog optical sensors*** that can differentiate between a white and black surface under the front of the robot. It also has two motors connected to its drive wheels that can be controlled using ***pulse-width modulation (PWM)***. Figure A-3 shows a block diagram of the LafBot's interface to the PIC 32 processor and its peripherals.

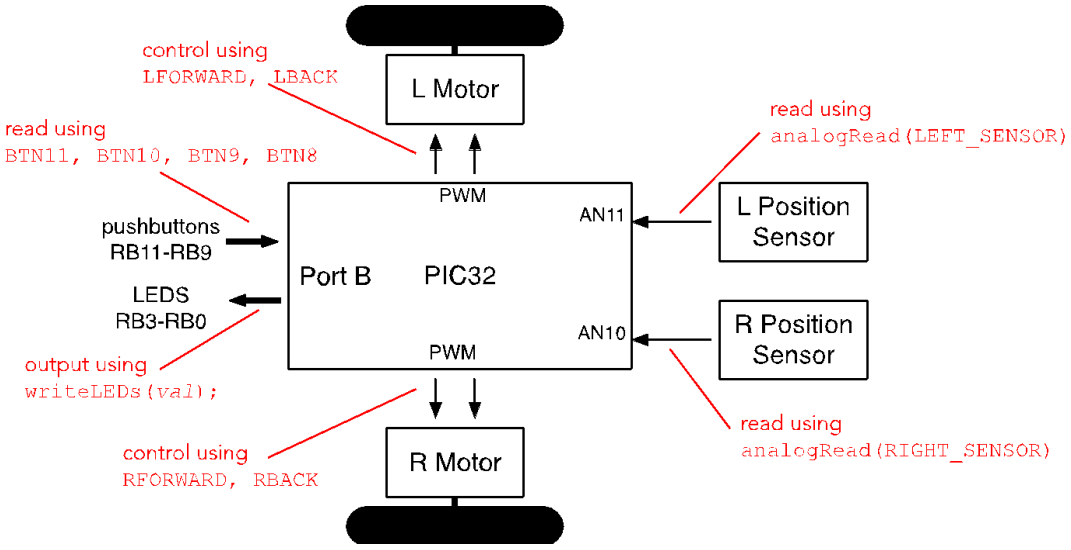


Figure A-3. LafBot Block Diagram

Interfacing with PIC32 Peripherals.

Microchip provides a number of libraries for configuring and accessing the peripherals on the PIC32 processor from a C program. For this course, most of the code useful for working with the LafBot can be found in a single header file named (**`ece212.h`**) that you can download from Moodle. You can include this file in your project using the directive:

```
#include "ece212.h"
```

Useful functions and variables defined in this header file are summarized below.

`ece212_setup(), ece212_lafbot_setup().` The first of these functions configures the PIC32 I/O to access the user pushbuttons and switches; the second configures the A/D and PWM outputs used by the LafBot to follow a line. Depending on the programming activity, your project should call one of these two functions to configure the board.

`delayms().` This function takes an integer parameter and causes the program to wait for the given number of milliseconds before executing the next statement.

`writeLEDs().` This function takes an integer parameter and uses the four least-significant bits of the integer to enable or disable the LafBot's LEDs: `RB3`, `RB2`, `RB1`, and `RB0`. For example, the following code snippet will turn on the LEDs `RB1` and `RB3` while turning `RB0` and `RB4` off.

```
writeLEDs(0xa);   //1010 in binary
writeLEDS(0x0);   // all off
writeLEDs(0xF);  // all on
writeLEDS(0x8);   // leftmost on
```

`BTN8, BTN9, BTN10, BTN11.` These macros are used like "read-only" variables that report the current status of the button as nonzero (pressed) or zero (not pressed). For example, the following code will sample and perform different actions depending on whether button BTN8 is pressed or not:

```
 if (BTN8){
   // action to be performed if button is pressed
 } else {
   // action to be performed if button is not pressed
 }
```

`RBACK, RFORWARD, LBACK, and LFORWARD.` These macros are used like "write-only" variables that set the speed for the two LafBot motors. The `RBACK` and `RFORWARD` variables control the right motor, whereas the `LBACK` and `LFORWARD` variables control the right motor. Each variable may be written with a value between `0` (off) and `0xFFFF` (maximum speed). Due to physical characteristics of the motors, a motor needs a value approximately `>=0x1FFF` to turn.

For example, the following code should make the robot move straight forward at maximum speed:

```
RBACK = 0;
RFORWARD = 0xFFFF;
LBACK = 0;
LFORWARD = 0xFFFF;
```

The following code should make the robot turn to the left:

```
RBACK = 0;
RFORWARD = 0x7fff;
LBACK = 0;
LFORWARD = 0x2fff;
```

Each motor should only be turned on in one direction at a time, as shown above. In other words, if `LFORWARD` is set to a non-zero value, `LBACK` must be set to a zero value and vice-versa. Similarly, if `RFORWARD` is set to a non-zero value, `RBACK` must be set to a zero value and vice-versa.

`analogRead()`. This function accepts a parameter indicating which analog pin to read. The macros `LEFT_SENSOR` and `RIGHT_SENSOR` are used to get the reading of the two sensors. For example, the following code reads from the left and right sensors and returns a 10-bit unsigned integer value.

```
left = analogRead(LEFT_SENSOR);
right = analogRead(RIGHT_SENSOR);
```

The LafBot is set up such that the sensors produce an analog voltage that ranges from approximately 0.8V to 1.2V depending on the amount of light reflected. The `analogRead()` function takes the measured voltage, `m`, relative to the maximum voltage (3.3V), and then scales this to a 10-bit unsigned integer value, according to the following formula.

$$m/3.3 * (2^{10}-1)$$

For example, the minimum voltage (0V) maps to the integer 0 and the maximum voltage (3.3V) maps to the integer value 1023. The voltage produced by the sensor reading is a subset of this range. For example, a possible sensor reading of 1V maps to the integer value 310.

A small reading means the sensor is over a light (or white) object, while a large reading means the sensor is over a dark (or black) object. The range of digitized values returned by the sensor may differ depending on whether the LafBot is powered by the battery or by the attached USB cable.

Appendix B: Moving the LafBot V2

Figure B-1 shows a closeup of the LafBot board that indicates the physical locations of the pushbuttons and LEDs.

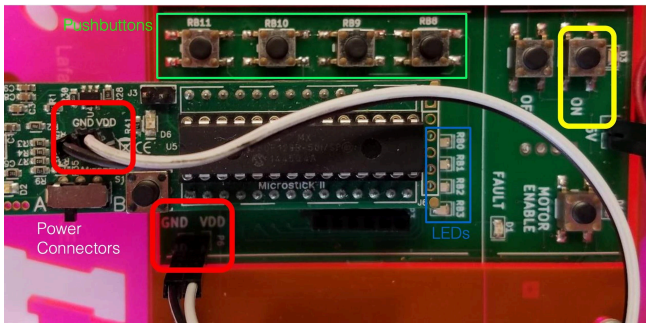
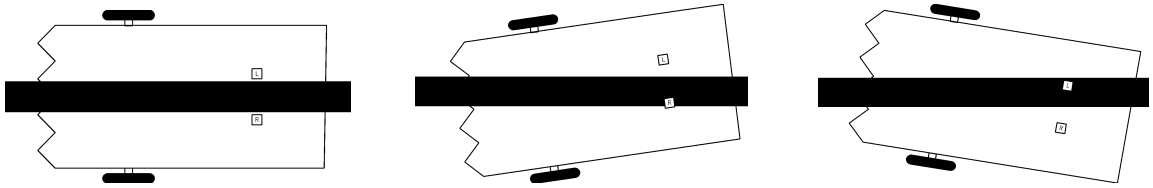


Figure B-1. LafBot Board Closeup

Controlling the LafBot Robot.

The LafBot has two drive wheels connected to motors that can be controlled by pulse-width modulation (PWM). A ball-bearing on the front of the robot acts as a front “wheel” so that the LafBot essentially a tricycle.

The LafBot will initially be placed on the track so that it is “on course” and the two sensors straddle the black line, as shown in Figure B-3. As the LafBot moves forward, it will eventually go off course either to te left or right, depending on the direction of the upcoming curve. This can happen either because the LafBot is not moving exactly in the right direction or because the track itself curves in one direction or another!



LafBot On Course ***LafBot Off Course, to Left*** ***LafBot Off Course, to Right***

Figure B-3. Steering the LafBot

When the LafBot is off of the track, your program can correct this error by changing the speed of the two wheels relative to each other. You may also want to vary the forward speed depending on whether the LafBot is following a straight segment or it is on a curvy region of the track.

The analog sensors on the LafBot will return different values depending on whether the sensor is over the black stripe or the white background (see **`analogRead()`** for more detail). The range of digitized values returned by the sensor will differ depending on whether the LafBot is powered by the battery or by the attached USB cable. For this reason, you should debug your code with the USB cable disconnected since that is how you will test the LafBot on the track. You may find it useful to use LEDs `RB0` and `RB3` to indicate whether your software detects the black stripe or white background to debug your sensor code before attempting to debug your overall program.